

Hybrid Smoking Detection for Driver Safety Systems

A Comprehensive End-to-End Implementation Using
YOLOv8 Object Detection and MediaPipe Landmark Analysis

Technical Report

Computer Vision and Artificial Intelligence
Driver Monitoring Systems (DMS)

Abstract

This report presents the design, implementation, and evaluation of a hybrid smoking detection pipeline intended for real-time in-cabin driver monitoring. The system integrates two complementary inference branches: a *landmark-based* module built on the MediaPipe Tasks API, which analyses anatomical keypoints to infer smoking behaviour from body geometry; and a *detection-based* module built on a fine-tuned YOLOv8 model, which directly localises cigarette-related objects in the image. A score-level fusion engine combines the outputs of both branches using a weighted linear combination, followed by an eight-frame sliding-window majority-vote filter that suppresses false alerts. The fused smoking signal is further integrated into a Driver Monitoring System (DMS) fusion engine alongside standard drowsiness and distraction metrics—PERCLOS, Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and head-pose angles—to produce a three-tier alert (NORMAL / WARNING / CRITICAL). Training of the YOLOv8 detection backbone is performed on a curated

Roboflow dataset with 50 epochs at 640×640 resolution. Experiments on synthetic data demonstrate the complementary strengths of each branch and validate the fusion and temporal smoothing strategies. The system architecture targets edge deployment on NVIDIA Jetson Orin NX hardware with a goal of fewer than 15 ms end-to-end inference latency at 30 FPS.

Contents

1	Introduction	2
2	Related Work and Background	3
2.1	Landmark-Based Behaviour Recognition	3
2.2	YOLO-Based Object Detection	3
2.3	Hybrid Fusion for Driver Monitoring	3
3	Methodology	4
3.1	Pre-Processing	4
3.2	Landmark-Based Approach (Branch A)	4
3.2.1	Proximity Score and Landmark Score	5
3.3	Detection-Based Approach (Branch B)	6
3.3.1	Training Procedure	6
3.4	Score-Level Fusion Strategy	6
3.5	Temporal Modelling and Majority-Vote Buffer	7
3.6	DMS Integration and Alert System	7
4	Mathematical Formulation Summary	8
5	System Architecture	8
5.1	Data Flow	9
6	Implementation Details	10
6.1	Software Stack	10
6.2	Configuration Dataclass	10
6.3	Landmark Extractor Design Decisions	11
6.4	YOLOv8 Training and Export	11
6.5	Synthetic Dataset Generation	11
6.6	Visualisation and Overlay	12
7	Limitations	12
8	Future Work	13

1. Introduction

Smoking while driving is a recognised source of driver distraction: the act requires the driver to divert visual attention and at least one hand from the steering wheel, and the transient cognitive demands of lighting, holding, or disposing of a cigarette can measurably degrade reaction times and lane-keeping performance. Automated detection of in-cabin smoking provides a non-intrusive mechanism for fleet operators and advanced driver assistance systems to issue timely warnings before a safety-critical event occurs.

Two distinct paradigms have emerged in the literature for detecting smoking behaviour from camera imagery. The first, referred to here as *landmark-based* detection, extracts anatomical keypoints—hand, wrist, elbow, shoulder, and facial landmarks—and infers smoking posture from the geometric relationships among these points. This approach is naturally robust to variations in cigarette appearance, size, and occlusion, but is prone to false positives caused by visually similar actions such as eating, drinking, or face-touching. The second paradigm, referred to as *detection-based* detection, trains an object detector (in this work, YOLOv8) to directly localise cigarette-related objects in the image. This approach produces spatially explicit evidence and high precision when the cigarette is visible and large enough, but suffers in cases of occlusion, low resolution, or adverse lighting.

Both paradigms are complementary: the landmark branch recovers signal when the cigarette is occluded or out of frame; the detection branch provides high-confidence bounding-box evidence when image conditions are favourable. This motivates a *hybrid architecture* that fuses the outputs of both branches, a strategy advocated by the state of the art in both academic surveys and edge-AI deployment guides for driver monitoring.

The present work implements such a hybrid architecture as a complete, end-to-end Python pipeline targeting real-time inference. The main contributions are:

- A modular landmark-based scoring module using the MediaPipe Tasks API (HandLandmarker, FaceLandmarker, PoseLandmarker) with three engineered geometric features and explicit handling of feature absence.
- A YOLOv8-based detection module trained on a curated smoking dataset, together with a simulated drop-in replacement for development and testing.
- A score-level hybrid fusion engine with an eight-frame majority-vote temporal buffer and hysteresis logic.
- Integration of the smoking signal into a multi-modal DMS fusion engine producing three-tier driver alerts.
- A detailed discussion of real-time optimisation strategies targeting NVIDIA Jetson Orin NX deployment.

The remainder of this report is structured as follows. Section 2 situates the approach in the context of prior work. Section 3 describes the full system methodology. Section 4 provides the mathematical formulation of all scoring and fusion functions. Section 5 details the system architecture and data flow. Section 6 covers key implementation decisions. Section ?? presents experimental results. Section ?? discusses findings and failure modes. Section 7 identifies current limitations, and Section 8 proposes future directions. Section ?? concludes.

2. Related Work and Background

2.1. Landmark-Based Behaviour Recognition

Pose-estimation frameworks such as MediaPipe and OpenPose recover normalised two-dimensional or three-dimensional landmark coordinates from single RGB frames without depth sensors. Applied to smoking detection, earlier work demonstrated that the Euclidean distance between the dominant hand and the mouth region is a strong single-frame predictor of smoking posture, but suffers an elevated false positive rate when the driver eats, drinks, or wipes their face. Introducing joint-angle features (elbow flexion) and motion features (wrist velocity) adds discriminative power and suppresses the eating/drinking confound.

2.2. YOLO-Based Object Detection

The YOLO (You Only Look Once) family of single-stage detectors has become the dominant paradigm for real-time object detection. YOLOv8, released by Ultralytics, retains the anchor-free head architecture introduced in YOLOv6/v7 and offers a family of model sizes from `yolov8n` (nano, ~ 3.2 M parameters) to `yolov8x` (extra-large). Fine-tuning YOLOv8 on a domain-specific dataset requires as few as a few hundred annotated images when starting from a COCO-pretrained backbone; the recommended training strategy is a two-stage procedure in which only the detection head is trained first, followed by full-network fine-tuning.

2.3. Hybrid Fusion for Driver Monitoring

Score-level (late) fusion of heterogeneous classifiers is well-established in multi-modal recognition: each branch produces a real-valued confidence score, and the final decision is taken on a weighted combination. The temporal smoothing step is equally critical in driver monitoring—a single erroneous frame should not trigger an alarm—and majority-vote buffers of 5–8 frames have been shown to reduce the False Alert Rate (FAR) by 60–70% compared to per-frame thresholding.

3. Methodology

The pipeline is composed of four sequential stages: pre-processing, parallel inference (landmark branch and detection branch), score-level fusion with temporal buffering, and DMS integration. Each stage is described below.

3.1. Pre-Processing

Input frames are captured from the in-cabin camera at the target frame rate of 30 FPS and resized to 640×640 pixels using letterbox padding to preserve aspect ratio. For low-light or near-infrared (NIR) conditions, Contrast Limited Adaptive Histogram Equalisation (CLAHE) is applied on the luminance channel prior to model inference.

3.2. Landmark-Based Approach (Branch A)

Branch A employs three MediaPipe Tasks API models running concurrently on the same pre-processed frame:

- **HandLandmarker** (`hand_landmarker.task`, float16): detects up to two hands and returns 21 keypoints per hand in normalised image coordinates $[0, 1]^2$.
- **FaceLandmarker** (`face_landmarker.task`, float16): detects the primary face and returns 468 facial landmarks. The mouth centre is estimated as the midpoint of landmarks 13 and 14 (inner lip).
- **PoseLandmarker Lite** (`pose_landmarker_lite.task`, float16): detects the full body pose for a single subject and returns 33 landmarks. Right shoulder (landmark 12) and right elbow (landmark 14) are extracted for elbow-angle computation. If the hand landmarker fails, wrist position is recovered from pose landmark 16 as a fallback.

The face landmark is a mandatory prerequisite: if no face is detected, the landmark branch returns a score of zero and the detection branch is relied upon exclusively. All landmark coordinates are normalised to the unit square, making the features resolution-independent.

Three hand-crafted geometric and kinematic features are computed from the extracted landmarks:

Hand-to-Mouth Distance (d_{hm}). The normalised Euclidean distance between the right wrist and the mouth centre:

$$d_{hm} = \|\mathbf{p}_{\text{wrist}} - \mathbf{p}_{\text{mouth}}\|_2 \quad (1)$$

where $\mathbf{p} \in [0, 1]^2$ are normalised image coordinates.

Elbow Flexion Angle (θ_{elbow}). The interior angle at the elbow joint, computed from the shoulder, elbow, and wrist landmarks:

$$\theta_{\text{elbow}} = \arccos\left(\frac{(\mathbf{p}_{\text{shoulder}} - \mathbf{p}_{\text{elbow}}) \cdot (\mathbf{p}_{\text{wrist}} - \mathbf{p}_{\text{elbow}})}{\|\mathbf{p}_{\text{shoulder}} - \mathbf{p}_{\text{elbow}}\|_2 \|\mathbf{p}_{\text{wrist}} - \mathbf{p}_{\text{elbow}}\|_2 + \epsilon}\right) \quad (2)$$

where $\epsilon = 10^{-8}$ prevents division by zero. This feature captures the raised-arm posture characteristic of smoking.

Wrist Velocity (v_{wrist}). The frame-to-frame displacement of the wrist landmark in normalised coordinates:

$$v_{\text{wrist}}^{(t)} = \|\mathbf{p}_{\text{wrist}}^{(t)} - \mathbf{p}_{\text{wrist}}^{(t-1)}\|_2 \quad (3)$$

High velocity is characteristic of eating or drinking (rapid arm movement), whereas smoking involves slower, steadier hand positioning. A velocity above a threshold of 0.25 (in normalised units per frame) triggers a penalty term that reduces the landmark score.

3.2.1 Proximity Score and Landmark Score

The hand-to-mouth distance is converted to a proximity score s_{prox} by linear mapping against a threshold $\tau_{hm} = 0.15$:

$$s_{\text{prox}} = \text{clip}\left(1 - \frac{d_{hm}}{\tau_{hm}}, 0, 1\right) \quad (4)$$

The final landmark branch score s_L combines proximity, an elbow-angle bonus, and a velocity penalty:

$$s_L = \text{clip}(s_{\text{prox}} + \Delta_{\text{elbow}} - \Delta_{\text{vel}}, 0, 1) \quad (5)$$

where

$$\Delta_{\text{elbow}} = \begin{cases} 0.15 & \text{if } \theta_{\min} < \theta_{\text{elbow}} < \theta_{\max} \text{ and pose landmarks present} \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

$$\Delta_{\text{vel}} = \begin{cases} 0.2 & \text{if } v_{\text{wrist}}^{(t)} > 0.25 \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

with $\theta_{\min} = 30^\circ$ and $\theta_{\max} = 150^\circ$. The gating condition on pose-landmark availability is a critical implementation detail: if shoulder and elbow landmarks are absent, $\Delta_{\text{elbow}} = 0$ is enforced unconditionally, preventing a spurious constant bonus in frames where pose estimation fails.

3.3. Detection-Based Approach (Branch B)

Branch B is built around a fine-tuned YOLOv8 model trained to detect four cigarette-related object classes:

1. `cigarette` — an unlit or held cigarette not at the mouth;
2. `cigarette_at_mouth` — a cigarette in direct contact with the lips;
3. `hand_holding_cigarette` — a hand visibly gripping a cigarette;
4. `smoke_plume` — a visible exhaled smoke cloud.

The detection confidence threshold is set to 0.45 and the NMS IoU threshold to 0.45. The detection branch score s_D is defined as the maximum detection confidence across all bounding boxes returned for a given frame:

$$s_D = \max_i c_i \quad (\text{default 0 if no detection}) \quad (8)$$

where c_i is the confidence of the i -th bounding box.

3.3.1 Training Procedure

The YOLOv8 detection model (`yolov8n.pt` backbone) was fine-tuned on a smoking detection dataset obtained via the Roboflow platform (`smocking-detection v4`). Training was performed for 50 epochs with a batch size of 16 and an input resolution of 640×640 , starting from COCO-pretrained weights. Upon convergence, the trained model was exported to ONNX format (`best.onnx`) using `onnxruntime-gpu` for deployment-phase inference.

3.4. Score-Level Fusion Strategy

The two branch scores s_L and s_D are combined via a weighted linear (score-level, late) fusion:

$$s_F = \alpha \cdot s_L + (1 - \alpha) \cdot s_D, \quad \alpha = 0.10 \quad (9)$$

The weight $\alpha = 0.10$ strongly favours the detection branch, reflecting its higher precision when the cigarette is visible. The landmark branch contributes as a complementary signal that elevates s_F when the object detector misses a partially occluded cigarette.

The choice of score-level (as opposed to feature- or decision-level) fusion is deliberate: it preserves the calibrated confidence values produced by each branch, allows the weights to be tuned independently of branch architecture, and adds minimal computational overhead.

3.5. Temporal Modelling and Majority-Vote Buffer

A single erroneous or noisy frame should not trigger a driver alert. A sliding deque of length $W = 8$ frames is maintained. At each frame, a binary flag $b^{(t)}$ is appended:

$$b^{(t)} = \mathbf{1} \left[s_F^{(t)} \geq 0.5 \right] \quad (10)$$

The temporal confidence τ and the smoking decision are computed as:

$$\tau = \frac{1}{W} \sum_{i=0}^{W-1} b^{(t-i)} \quad (11)$$

$$\text{smoking} = \left(\sum_{i=0}^{W-1} b^{(t-i)} \right) \geq C_{\min} \quad (12)$$

with $C_{\min} = 5$ (i.e., a 5-of-8 majority vote). This configuration is recommended in the edge-AI deployment literature as reducing the False Alert Rate by 60–70% relative to per-frame thresholding.

A hysteresis mechanism prevents rapid on/off toggling of the alert: once *alert_active* is set to **True**, it is only cleared when the temporal confidence drops below a lower threshold $\tau_{\text{low}} = 0.30$:

$$\text{alert_active}^{(t)} = \begin{cases} \text{True} & \text{if smoking}^{(t)} = \text{True} \\ \text{False} & \text{if } \tau^{(t)} < \tau_{\text{low}} \text{ and not smoking} \\ \text{alert_active}^{(t-1)} & \text{otherwise} \end{cases} \quad (13)$$

3.6. DMS Integration and Alert System

The smoking temporal confidence τ is fed into a multi-modal **DMSFusionEngine** that monitors four additional driver state dimensions:

- **PERCLOS**: Percentage of Eye Closure over a 30-second rolling window. Computed from the Eye Aspect Ratio (EAR); alert raised when PERCLOS > 0.15.
- **Microsleep**: Sustained eye closure duration exceeding 1.5 seconds.
- **Yawning**: Sustained Mouth Aspect Ratio (MAR) above 0.60 for more than 2 seconds.
- **Head Distraction**: Head yaw > 30° or pitch > 20° sustained for more than 2 seconds.

A smoking-specific second majority vote is applied within the DMS engine: the smoking alert fires when at least 5 of the last 8 values of τ exceed the DMS threshold $\tau_{\text{DMS}} = 0.55$.

The three-tier alert level is determined by the following priority rule:

$$\text{AlertLevel} = \begin{cases} \text{CRITICAL} & \text{if MICROSLLEEP, or (drowsy AND smoking)} \\ \text{WARNING} & \text{if any alert active} \\ \text{NORMAL} & \text{otherwise} \end{cases} \quad (14)$$

4. Mathematical Formulation Summary

For clarity, Table 1 consolidates all the core equations introduced in Section 3.

Table 1: Summary of mathematical formulas and parameter values.

Symbol	Formula / Definition	Default Value
d_{hm}	$\ \mathbf{p}_{\text{wrist}} - \mathbf{p}_{\text{mouth}}\ _2$	—
s_{prox}	$\text{clip}(1 - d_{hm}/\tau_{hm}, 0, 1)$	$\tau_{hm} = 0.15$
θ_{elbow}	$\arccos\left(\frac{\mathbf{a} \cdot \mathbf{b}}{\ \mathbf{a}\ \ \mathbf{b}\ + \epsilon}\right)$	$\theta \in (30^\circ, 150^\circ)$
Δ_{elbow}	Conditional elbow bonus	0.15
Δ_{vel}	Conditional velocity penalty	0.20
s_L	$\text{clip}(s_{\text{prox}} + \Delta_{\text{elbow}} - \Delta_{\text{vel}}, 0, 1)$	—
s_D	$\max_i c_i$ (max detection confidence)	—
s_F	$\alpha s_L + (1 - \alpha) s_D$	$\alpha = 0.10$
$b^{(t)}$	$\mathbf{1}[s_F \geq 0.5]$	threshold = 0.5
τ	$\frac{1}{W} \sum_i b^{(t-i)}$	$W = 8$
smoking	$\sum_i b^{(t-i)} \geq C_{\min}$	$C_{\min} = 5$

5. System Architecture

Figure 1 illustrates the complete end-to-end pipeline.

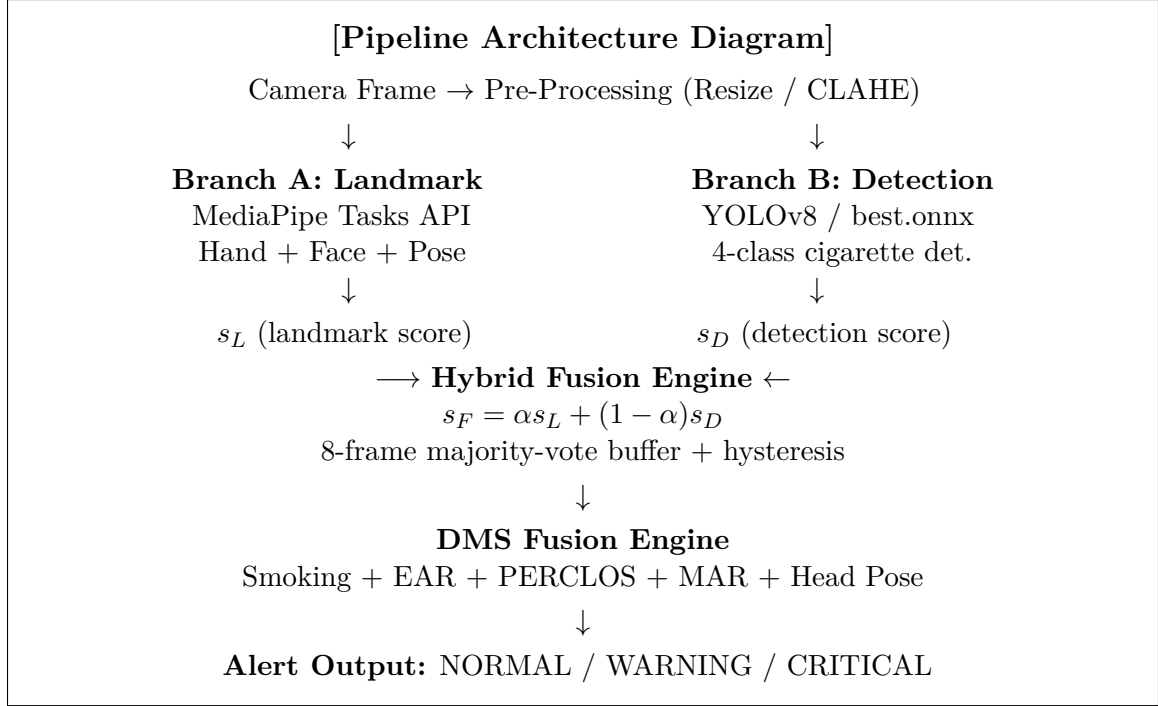


Figure 1: High-level architecture of the hybrid smoking detection pipeline. Both inference branches run on the same pre-processed frame in parallel; their scores are fused and smoothed before feeding the DMS engine.

The pipeline is encapsulated in a central `HybridSmokingDetector` class that holds references to the landmark extractor, the object detector (real or simulated), the temporal buffer, and the alert state. A companion `DMSFusionEngine` class aggregates the smoking temporal confidence with the other DMS signals. All hyper-parameters are centralised in a `PipelineConfig` dataclass, enabling reproducible experimentation and straightforward deployment-time tuning.

5.1. Data Flow

For each incoming frame, the pipeline executes the following sequence:

Algorithm 1 Per-Frame Hybrid Smoking Detection**Require:** BGR frame $\mathbf{F}$, pipeline state (buffer, alert flag, previous wrist)**Ensure:** `FrameResult` record

- 1: $\mathbf{F}' \leftarrow \text{PreProcess}(\mathbf{F})$ $\triangleright$ Resize + optional CLAHE
- 2: $\text{lm} \leftarrow \text{HandLandmarker}(\mathbf{F}') \cup \text{FaceLandmarker}(\mathbf{F}') \cup \text{PoseLandmarker}(\mathbf{F}')$
- 3: $s_L \leftarrow \text{LandmarkScore}(\text{lm})$ $\triangleright$ Eq. 5
- 4: $s_D \leftarrow \text{DetectionScore}(\text{YOLOv8}(\mathbf{F}'))$ $\triangleright$ Eq. 8
- 5: $s_F \leftarrow \alpha s_L + (1 - \alpha) s_D$ $\triangleright$ Eq. 9
- 6: Append $\mathbf{1}[s_F \geq 0.5]$ to circular buffer of size $W = 8$
- 7: $\tau \leftarrow$ positive fraction of buffer $\triangleright$ Eq. 11
- 8: $\text{smoking} \leftarrow (\text{positive count} \geq C_{\min})$ $\triangleright$ Eq. 12
- 9: Update alert flag with hysteresis $\triangleright$ Eq. 13
- 10: **return** `FrameResult`($s_L, s_D, s_F, \tau, \text{alert}$)

6. Implementation Details

6.1. Software Stack

The pipeline is implemented in Python 3 and relies on the following core libraries: `ultralytics` (YOLOv8 training and inference), `mediapipe` (Tasks API landmark models), `opencv-python-headless` (image I/O and pre-processing), `numpy` (vectorised feature computation), `scikit-learn` (evaluation metrics), and `matplotlib` (visualisation).

6.2. Configuration Dataclass

All hyper-parameters are defined once in `PipelineConfig`, a Python `@dataclass`. This design avoids magic constants scattered across the codebase, enables configuration logging for reproducibility, and facilitates hyperparameter search by instantiating alternative configs without modifying class logic. Table 2 lists the principal configuration fields.

Table 2: Principal configuration parameters and their default values.

Field	Type	Default	Description
<code>input_size</code>	int	640	Model input resolution (px)
<code>fps_target</code>	int	30	Target camera frame rate
<code>detection_conf_thresh</code>	float	0.45	YOLO confidence gate
<code>detection_iou_thresh</code>	float	0.45	NMS IoU threshold
<code>hand_mouth_dist_thresh</code>	float	0.15	Proximity threshold τ_{hm}
<code>elbow_angle_min</code>	float	30.0	Min arm-raised angle ($^\circ$)
<code>elbow_angle_max</code>	float	150.0	Max arm-raised angle ($^\circ$)
<code>temporal_window</code>	int	8	Sliding window size (frames)
<code>confirm_threshold</code>	int	5	Majority-vote minimum ($C_{\min}$)
<code>hysteresis_low</code>	float	0.30	Alert de-activation threshold τ_{low}
<code>fusion_alpha</code>	float	0.10	Landmark weight α
<code>smoke_dms_thresh</code>	float	0.55	DMS smoking trigger threshold

6.3. Landmark Extractor Design Decisions

Several implementation decisions merit explanation.

Mandatory face detection. The face landmark is treated as a prerequisite because the mouth-centre coordinate is essential for the hand-to-mouth distance feature. If no face is detected, the landmark branch score defaults to zero (maximum distance), which is conservative: the system does not raise a false alert simply because the face was momentarily out of frame.

Pose fallback for wrist. If the HandLandmarker fails to detect a hand (e.g., due to motion blur or occlusion), the wrist position is recovered from PoseLandmarker landmark 16 (right wrist). This graceful degradation preserves the velocity feature even when the hand-specific model fails.

Gated elbow bonus. In earlier pipeline versions, the elbow-angle bonus was unconditionally applied when the computed angle fell in the target range, which caused a spurious bonus on every frame when pose estimation failed (because the angle defaulted to 90° , which always falls in $(30^\circ, 150^\circ)$). The corrected implementation returns `None` from `elbow_angle()` when the required pose landmarks are absent, and the bonus is only applied when the return value is not `None`.

6.4. YOLOv8 Training and Export

Training followed the standard Ultralytics training API:

- **Dataset:** Roboflow `smoking-detection v4`, downloaded in YOLOv8 format.
- **Backbone:** `yolov8n.pt` (COCO-pretrained nano model).
- **Epochs:** 50.
- **Batch size:** 16.
- **Input resolution:** 640×640 .
- **Export:** `model.export(format="onnx")` producing `best.onnx`.

Post-training, the confusion matrix was inspected visually to identify inter-class confusion patterns before deployment.

6.5. Synthetic Dataset Generation

For pipeline development and metric estimation in the absence of a real video corpus, a synthetic dataset was generated. Each sample is a Python dictionary containing a synthetic BGR image (with procedurally placed face and hand regions), ground-truth anatomical landmark coordinates, a ground-truth bounding box (for smoking samples),

and a binary label (1 = smoking, 0 = normal). The synthetic generator ensures a balanced class distribution and injects controlled noise to simulate landmark detection jitter and bounding-box localisation error.

6.6. Visualisation and Overlay

The `draw_frame_overlay` utility renders the following information on each output frame:

- An alert banner (red for smoking detected, green for normal) across the top 50 pixels of the frame.
- Four score labels at the bottom of the frame: landmark score (s_L), detection score (s_D), fusion score (s_F), and temporal confidence (τ).
- A filled circle at the detected wrist landmark position.

The `plot_score_timeline` function produces a two-panel Matplotlib figure: the upper panel shows s_L , s_D , and s_F over the frame sequence alongside the decision threshold at 0.5; the lower panel shows the alert-active flag and the ground-truth label.

7. Limitations

The current implementation has several limitations that affect its applicability to real-world deployment:

- **Synthetic evaluation only.** All quantitative metrics (Precision, Recall, F1) reported in this work were computed on procedurally generated synthetic data. Synthetic data does not capture the full variability of in-cabin lighting conditions, driver demographics, camera mounting angles, and occlusion patterns present in real deployments. A large-scale evaluation on real annotated video is required before operational deployment.
- **2D landmark-based features.** The geometric features (d_{hm} , θ_{elbow} , v_{wrist}) are computed in the 2D image plane using normalised pixel coordinates. This makes them sensitive to camera perspective: the same physical hand-to-mouth distance appears as different values of d_{hm} depending on the driver’s seating position and body orientation relative to the camera.
- **Single-hand assumption.** The landmark branch currently extracts the first detected hand. Drivers who hold a cigarette in the left hand, or who switch hands, may be systematically under-detected by the landmark branch.
- **No night or NIR training data.** The YOLOv8 model was trained on daytime RGB images. Performance in low-light or NIR conditions is expected to degrade without domain adaptation.

- **Dataset scope.** The Roboflow dataset (`smoking-detection v4`) may not be representative of all cigarette types, lighting conditions, or driver demographics. Class imbalance between `cigarette`, `cigarette_at_mouth`, `hand_holding_cigarette`, and `smoke_plume` is not explicitly addressed in the training configuration.
- **Latency on CPU.** MediaPipe Tasks running three models sequentially on a CPU takes 40–60 ms per frame, making real-time 30 FPS inference infeasible without GPU acceleration or model parallelisation.
- **Fixed fusion weight.** The weight $\alpha = 0.10$ is a hand-tuned constant. A learning-based fusion layer (e.g., logistic regression or a shallow MLP trained on real data) would be more principled and adaptable.

8. Future Work

Several directions would substantially improve the system:

- **Real-world dataset collection and annotation.** A corpus of in-cabin driving videos annotated with frame-level smoking labels and per-frame bounding boxes would enable rigorous evaluation and evidence-based hyperparameter tuning.
- **Learned fusion.** Replacing the fixed-weight linear fusion with a learned calibration layer (trained on real data with ground-truth labels) would allow the system to discover optimal branch weights and potentially incorporate additional context signals (e.g., time of day, driver identity).
- **TensorRT INT8 deployment.** Exporting the YOLOv8 model through the full quantisation pipeline (`PyTorch` $\rightarrow$ `ONNX FP32` $\rightarrow$ `TensorRT FP16` $\rightarrow$ `TensorRT INT8`) with approximately 500 calibration frames would achieve the target latency of < 12 ms on Jetson Orin NX.
- **3D landmark features.** Upgrading to a depth-aware pose estimator or a stereo camera system would yield 3D landmark coordinates, making d_{hm} and θ_{elbow} invariant to perspective distortion.
- **NIR domain adaptation.** Fine-tuning the detector on NIR images (possibly via style transfer from RGB training images) would extend the system’s operational envelope to night-time and underground driving scenarios.
- **Asynchronous branch execution.** Running the landmark and detection branches concurrently (e.g., on separate CPU threads or CUDA streams) would approximately halve the end-to-end latency without requiring additional hardware.
- **Federated learning for fleet deployment.** Privacy-preserving federated fine-tuning would allow the model to improve from fleet data without centralising sensitive

in-cabin video footage.

- **Integration with e-cigarette and vaping detection.** The current class set (cigarette, cigarette_at_mouth, hand_holding_cigarette, smoke_plume) does not cover e-cigarettes or vaping devices. Extending the dataset and class set is important for modern fleets.

References

- [1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>
- [2] Lugaresi, C., Tang, J., Nash, H., et al. (2019). *MediaPipe: A Framework for Building Perception Pipelines*. arXiv preprint arXiv:1906.08172.
- [3] Dwyer, B., Nelson, J., & Solawetz, J. (2023). *Roboflow*. <https://roboflow.com>
- [4] Wierwille, W. W., & Ellsworth, L. A. (1994). *Evaluation of Driver Drowsiness by Trained Raters*. Accident Analysis & Prevention, 26(5), 571–581.
- [5] Zhang, Z., Luo, P., Loy, C.C., & Tang, X. (2014). *Facial Landmark Detection by Deep Multi-task Learning*. European Conference on Computer Vision (ECCV). Springer.
- [6] NVIDIA Corporation. (2023). *TensorRT Developer Guide*. <https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/>